

Special Participation B:

Windsurf SWE-1 on HW07 Coding Tasks

Aryan Bansal

Abstract

This document evaluates the performance of Windsurf SWE-1, an AI coding assistant, on the EECS 182 Fall 2025 Homework 7 coding tasks. The homework consists of four Jupyter notebooks covering autoencoders, RNN implementations and gradient analysis, RNN-based name classification, and graph clustering. We analyze each attempted implementation, comparing it against the reference solutions and assessing correctness, style, and understanding of the underlying concepts.

Contents

1	Overview and Methodology	2
2	Notebook 1: Autoencoders	2
2.1	Task 1: Decoder Construction (<code>_build_decoder</code>)	2
2.2	Task 2: Forward Pass	2
2.3	Task 3: Loss Function	3
2.4	Task 4: Denoising Autoencoder Training Step	3
2.5	Task 5: Masked Autoencoder Training Step	4
2.6	Task 6: Model Accuracy Evaluation	4
2.7	Task 7: Visualization Function	5
2.8	Task 8: MNIST Hyperparameters	5
2.9	Autoencoder Summary	6
3	Notebook 2: RNN and Gradient Analysis	6
3.1	Task 1: RNNLayer Implementation	6
3.2	Task 2: RecurrentRegressionModel	7
3.3	Task 3: Loss Function	7
3.4	Task 4: Multi-layer RNN	8
3.5	Task 5: Gradient Visualizer Weight Scaling	8
3.6	Task 6: LSTMLayer (Not shown in Windsurf)	9
3.7	RNN and Gradient Summary	9
4	Notebook 3: RNN Last Name Classification	9
4.1	RNN Last Name Summary	10
5	Notebook 4: Graph Clustering	11
5.1	Graph Clustering Summary	12

6	Overall Assessment	12
6.1	Quantitative Summary	12
6.2	Key Observations	13
6.3	Estimated Score Impact	13
6.4	Recommendations for AI-Assisted Homework	13

1 Overview and Methodology

The evaluation covers four notebooks:

1. `q.autoencoder.ipynb` — Vanilla, Denoising, and Masked Autoencoders
2. `q.rnn_and_grad.ipynb` — RNN/LSTM implementation and gradient visualization
3. `q.rnn_last_name.ipynb` — Character-level RNN for name origin classification
4. `q.graph_clustering.ipynb` — Spectral clustering with graph representations

Each solution is presented in a colored box indicating correctness:

- **Green** — Correct or functionally equivalent
- **Orange** — Partially correct with minor issues
- **Red** — Incorrect or fundamentally flawed

2 Notebook 1: Autoencoders

2.1 Task 1: Decoder Construction (`_build_decoder`)

✓ Correct

Windsurf Implementation:

```
1 def _build_decoder(self):
2     layers = []
3     prev_size = self.hidden_sizes[-1]
4     for i in range(len(self.hidden_sizes)-2, -1, -1):
5         layers.append(nn.Linear(prev_size, self.hidden_sizes[i]))
6         layers.append(self.activation_cls())
7         prev_size = self.hidden_sizes[i]
8     layers.append(nn.Linear(prev_size, self.input_size))
9     return nn.Sequential(*layers)
```

Analysis: This implementation correctly constructs a symmetric decoder. The solution iterates through the hidden sizes in reverse order, appending linear layers and activations. The approach using index-based iteration is functionally equivalent to the reference solution which uses `reversed(self.hidden_sizes[:-1])`.

Reference Solution Comparison: Uses `for size in reversed(self.hidden_sizes[:-1])` which is more Pythonic but produces identical network architecture.

2.2 Task 2: Forward Pass

✓ Correct

Windsurf Implementation:

```
1 def forward(self, x: torch.Tensor) -> torch.Tensor:
2     z = self.encoder(x)
3     x_hat = self.decoder(z)
4     return x_hat
```

Analysis: Identical to the reference solution. Correctly passes input through encoder to get latent representation, then through decoder for reconstruction.

2.3 Task 3: Loss Function

~ Minor Difference

Windsurf Implementation:

```
1 def loss(self, x: torch.Tensor, x_hat: torch.Tensor) -> torch.Tensor:
2     return F.mse_loss(x_hat, x, reduction='mean')
```

Reference Solution:

```
1 return F.mse_loss(x, x_hat, reduction="sum") / x.size(0)
```

Analysis: Both compute MSE loss, but with different reduction strategies:

- Windsurf uses `reduction='mean'` which averages over all elements
- Reference uses `reduction='sum'` divided by batch size, averaging only over the batch dimension

These are **not identical** for multi-dimensional inputs. The reference solution computes per-sample total reconstruction error averaged over the batch, while Windsurf averages over all dimensions. This could affect gradient magnitudes and convergence, though the model should still train.

2.4 Task 4: Denoising Autoencoder Training Step

✓ Correct

Windsurf Implementation:

```
1 def train_step(self, x: torch.Tensor, optimizer) -> torch.Tensor:
2     noise = torch.randn_like(x) * self.noise_std
3     noisy_x = x + noise
4     x_hat = self(noisy_x)
5     loss = self.loss(x, x_hat)
6     optimizer.zero_grad()
7     loss.backward()
8     optimizer.step()
9     return loss
```

Analysis: Correctly implements denoising autoencoder training:

1. Generates Gaussian noise scaled by `noise_std`
2. Adds noise to create corrupted input
3. Reconstructs from noisy input
4. Computes loss against *clean* input (correct!)
5. Performs backpropagation

2.5 Task 5: Masked Autoencoder Training Step

✓ Correct

Windsurf Implementation:

```
1 def train_step(self, x: torch.Tensor, optimizer) -> torch.Tensor:
2     mask = torch.rand_like(x) > self.mask_prob
3     masked_x = x * mask
4     x_hat = self(masked_x)
5     loss = self.loss(x, x_hat)
6     optimizer.zero_grad()
7     loss.backward()
8     optimizer.step()
9     return loss
```

Analysis: Correctly implements masked autoencoder:

- Creates binary mask with probability `mask_prob` of being 0
- Zeros out masked positions
- Reconstructs from masked input
- Loss computed against original unmasked input

Minor Note: Uses `torch.rand_like(x)` vs reference's `torch.rand(x.shape, device=x.device)` — functionally equivalent.

2.6 Task 6: Model Accuracy Evaluation

~ Minor Issues

Windsurf Implementation:

```
1 def _get_model_accuracy(self, classifier: nn.Module, split="test") -> float:
2     self.model.eval()
3     num_correct = 0
4     num_total = 0
5     num_batches = self.dataset.num_test_samples // self.eval_batch_size
6     with torch.no_grad():
7         for batch_idx in range(num_batches):
8             x, y = self.dataset.get_batch(batch_idx, split=split)
9             x, y = x.cuda(), y.cuda()
10            features = self.model.encode(x)
11            logits = classifier(features)
12            preds = torch.argmax(logits, dim=1)
13            num_correct += (preds == y).sum().item()
14            num_total += y.size(0)
15     return (num_correct / num_total) * 100
```

Analysis: The implementation is *mostly correct* but has one notable difference:

- + Uses `torch.no_grad()` context (good practice for evaluation)
- + Correctly encodes inputs and classifies with the linear probe

- The reference solution hard-codes `split="test"` in the batch retrieval, ignoring the `split` parameter. Windsurf’s version is actually *more correct* by using the parameter.

2.7 Task 7: Visualization Function

~ Minor Differences

Windsurf Implementation:

```
1 def plot_single(values, feats_dim):
2     values = np.array(values)
3     mean = values.mean(axis=0)
4     min_vals = values.min(axis=0)
5     max_vals = values.max(axis=0)
6     x = np.arange(1, len(mean) + 1)
7     plt.plot(x, mean, label=f'Dim={feats_dim}')
8     plt.fill_between(x, min_vals, max_vals, alpha=0.3)
```

Issues identified:

1. X-axis starts at 1 instead of 0 (epochs should start at 0)
2. Missing `marker='o'` for data points
3. Label format differs: `Dim={feats_dim}` vs `d = {feats_dim:3d}`
4. Uses `alpha=0.3` vs reference’s `alpha=0.4` (minor)

2.8 Task 8: MNIST Hyperparameters

~ Different but Reasonable

Windsurf Suggestion:

```
1 hidden_sizes=[256, 64, 32] # Bottleneck size of 32
2 activation="ReLU"
3 # Uses vanilla Autoencoder
```

Reference Solution:

```
1 hidden_dims=[100, 100]
2 activation="ReLU"
3 noise_std=0.5, mask_prob=0.25
4 # Uses MaskedAutoencoder achieving 79.95% accuracy
```

Analysis: Windsurf suggested a vanilla autoencoder with a progressively smaller bottleneck. The reference uses a **MaskedAutoencoder** with wider hidden layers. The masked autoencoder’s regularization effect likely helps achieve the 79% accuracy target.

2.9 Autoencoder Summary

Task	Status	Would Pass Tests?
Decoder construction	✓ Correct	Yes
Forward pass	✓ Correct	Yes
Loss function	~ Minor diff	Likely yes
Denoising train step	✓ Correct	Yes
Masked train step	✓ Correct	Yes
Model accuracy	✓ Correct	Yes
Visualization	~ Minor issues	Visual differences
MNIST hyperparams	~ Different	Unknown

3 Notebook 2: RNN and Gradient Analysis

3.1 Task 1: RNNLayer Implementation

× Structural Mismatch

Windsurf Implementation:

```
1 class RNNLayer(nn.Module):
2     def __init__(self, input_size, hidden_size, nonlinearity=th.tanh):
3         super().__init__()
4         self.input_size = input_size
5         self.hidden_size = hidden_size
6         self.nonlinearity = nonlinearity
7         self.W_hh = nn.Linear(hidden_size, hidden_size, bias=False)
8         self.W_xh = nn.Linear(input_size, hidden_size, bias=True)
9
10    def forward(self, x):
11        batch_size = x.size(0)
12        seq_len = x.size(1)
13        h_prev = th.zeros(batch_size, self.hidden_size, device=x.device)
14        h_list = []
15        for t in range(seq_len):
16            x_t = x[:, t, :]
17            h_t = self.nonlinearity(self.W_hh(h_prev) + self.W_xh(x_t))
18            h_list.append(h_t)
19            h_prev = h_t
20        self.store_h_for_grad(h_list)
21        all_h = th.stack(h_list, dim=1)
22        last_h = h_list[-1] if h_list else h_prev
23        return all_h, last_h
```

Reference Solution:

```
1 self.w = nn.Linear(input_size + hidden_size, hidden_size, bias=True)
2 # ...
3 for i in range(x.shape[1]):
4     in_value = th.cat([x[:, i], h_i], dim=1)
5     h_i = self.nonlinearity(self.w(in_value))
```

Critical Analysis:

1. **Different parameter structure:** Windsurf uses two separate linear layers (W_{hh} , W_{xh}), while reference uses a single combined layer w .
2. **Mathematically equivalent:** Both compute $h_t = \sigma(W_{hh}h_{t-1} + W_{xh}x_t + b)$
3. **Test failure:** The test suite checks `model.state_dict().keys()` and expects exactly 18 parameters with specific names. Windsurf's version would have different parameter names and would fail the assertions.

3.2 Task 2: RecurrentRegressionModel

✓ Correct

Windsurf Implementation:

```

1 class RecurrentRegressionModel(nn.Module):
2     def __init__(self, recurrent_net, output_dim=1):
3         super().__init__()
4         self.recurrent_net = recurrent_net
5         self.output_dim = output_dim
6         self.linear = nn.Linear(recurrent_net.hidden_size, output_dim)
7
8     def forward(self, x):
9         all_h, _ = self.recurrent_net(x)
10        out = self.linear(all_h)
11        return out, all_h

```

Analysis: Essentially identical to reference. Uses `self.linear` vs `self.w` for the output layer name, but functionally equivalent.

3.3 Task 3: Loss Function

✓ Correct

Windsurf Implementation:

```

1 def loss_fn(pred, y, last_timestep_only=False):
2     if last_timestep_only:
3         loss = F.mse_loss(pred[:, -1:], y[:, -1:])
4     else:
5         loss = F.mse_loss(pred, y)
6     return loss

```

Analysis: Correct implementation. Minor difference: Windsurf uses slicing `[:, -1:]` which preserves dimensionality, while reference uses `[:, -1]` then creates `MSELoss()`. Both are correct.

3.4 Task 4: Multi-layer RNN

✓ Correct

Windsurf Implementation:

```
1 class RNN(nn.Module):
2     def __init__(self, input_size, hidden_size, num_layers):
3         super().__init__()
4         # ...
5         self.layers = nn.ModuleList()
6         for i in range(num_layers):
7             in_size = input_size if i == 0 else hidden_size
8             self.layers.append(RNNLayer(in_size, hidden_size))
9
10    def forward(self, x):
11        # ...
12        for layer in self.layers:
13            all_h, last_h = layer(current_input)
14            # ...
15        last_step_h = th.stack(last_hiddens, dim=0)
16        return layer_outputs[-1], last_step_h
```

Analysis: Correct architecture for multi-layer RNN. Properly handles varying input sizes for first vs subsequent layers.

3.5 Task 5: Gradient Visualizer Weight Scaling

~ Partially Correct

Windsurf Implementation:

```
1 def update_plots(self, weight_val=1.0, bias_val=1.0):
2     w_dict = copy.deepcopy(self.original_weights)
3     for k in w_dict:
4         if 'weight' in k:
5             w_dict[k] = w_dict[k] * weight_val
6         elif 'bias' in k:
7             w_dict[k] = w_dict[k] * bias_val
8     # ...
9     if isinstance(self.rnn, RNNLayer):
10        W_hh = self.rnn.W_hh.weight
11        W_xh = self.rnn.W_xh.weight
12        W = th.cat([W_hh, W_xh], dim=1)
13        biggest_eig = biggest_eig_magnitude(W)
```

Issues:

1. The eigenvalue computation concatenates both weight matrices, which is **incorrect**. Only W_{hh} affects gradient explosion/vanishing through time.
2. The reference correctly extracts only the hidden-to-hidden portion: `W = self.rnn.w.weight[:, -h_len:]`
3. Default parameter values differ: Windsurf uses `weight_val=1.0` vs reference's `weight_val=0`

3.6 Task 6: LSTMLayer (Not shown in Windsurf)

Windsurf did not provide an LSTM implementation in the captured conversation, though the reference solution implements a complete LSTM with proper gate computations.

3.7 RNN and Gradient Summary

Task	Status	Would Pass Tests?
RNNLayer	× Wrong structure	No (param names)
RecurrentRegressionModel	✓ Correct	Yes
Loss function	✓ Correct	Yes
Multi-layer RNN	✓ Correct	Yes*
Gradient Visualizer	~ Partial	Partial
LSTMLayer	Not provided	N/A

*Depends on RNNLayer being correct first.

4 Notebook 3: RNN Last Name Classification

× Completely Different Approach

Windsurf Implementation:

```
1 class CharRNN(nn.Module):
2     def __init__(self, vocab_size, embed_dim, hidden_dim, num_layers=1):
3         super().__init__()
4         self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
5         self.rnn = nn.RNN(embed_dim, hidden_dim, num_layers=num_layers,
6                             batch_first=True)
7         self.fc = nn.Linear(hidden_dim, vocab_size)
8
9     def forward(self, x, hidden=None):
10        embedded = self.embedding(x)
11        output, hidden = self.rnn(embedded, hidden)
12        output = self.fc(output)
13        return output, hidden
14
15    def generate(self, start_char, char_to_idx, idx_to_char,
16                max_len=20, temperature=1.0):
17        # ... generation code ...
```

Also provided standalone training functions and data preprocessing that don't match the notebook structure.

Critical Issues:

1. **Wrong task interpretation:** Windsurf implemented a *name generation* model (language model predicting next character), not the required *classification* model.
2. **Missing required class:** The task requires implementing `RecurrentClassifier` with:
 - Embedding layer
 - RNN/LSTM stack with `batch_first=True`

- Extraction of last non-padded hidden state using `torch.gather`
- Classification head outputting `n_categories` logits

3. Output mismatch:

- Windsurf: `fc` outputs `vocab_size` (next character prediction)
- Required: output layer outputs `n_categories` (language classification)

4. **Missing key component:** The reference uses `torch.gather` to extract hidden states at variable-length sequence positions:

```
1 out = torch.gather(
2     rnn_out, 1,
3     last_pos.view(-1, 1, 1).expand(-1, 1, self.rnn_size)
4 ).squeeze(1)
```

Windsurf's generation-focused approach doesn't handle variable-length inputs for classification.

Reference Solution Structure:

```
1 class RecurrentClassifier(nn.Module):
2     def __init__(self, vocab_size, rnn_size, n_categories,
3                 num_layers=1, dropout=0.0, model_type='lstm'):
4         self.embedding = nn.Embedding(vocab_size, rnn_size)
5         self.rnn = nn.RNN(rnn_size, rnn_size, num_layers=num_layers,
6                          dropout=dropout, batch_first=True)
7         self.drop = nn.Dropout(dropout)
8         self.output = nn.Linear(rnn_size, n_categories)
9
10    def forward(self, x, last_pos):
11        embeds = self.embedding(x)
12        rnn_out, _ = self.rnn(embeds)
13        # Extract hidden state at last_pos for each sequence
14        out = torch.gather(rnn_out, 1,
15                          last_pos.view(-1, 1, 1).expand(-1, 1, self.rnn_size)).squeeze(1)
16        out = self.drop(out)
17        logits = self.output(out)
18        return logits
```

4.1 RNN Last Name Summary

Task	Status	Would Pass Tests?
RecurrentClassifier	× Wrong task	No
Embedding layer	✓ Present but wrong use	No
RNN/LSTM stack	~ Present but wrong output	No
Last position extraction	× Missing	No
Classification head	× Wrong output dim	No
Hyperparameter tuning	Not applicable	N/A

5 Notebook 4: Graph Clustering

× Entirely Wrong Approach

Windsurf Implementation:

```
1 class GraphAutoencoder(nn.Module):
2     def __init__(self, num_features, hidden_dims, num_clusters):
3         super().__init__()
4         # Encoder
5         self.conv1 = GCNConv(num_features, hidden_dims[0])
6         self.conv2 = GCNConv(hidden_dims[0], hidden_dims[1])
7
8         # Decoder
9         self.decoder = nn.Sequential(
10             nn.Linear(hidden_dims[1], hidden_dims[0]),
11             nn.ReLU(),
12             nn.Linear(hidden_dims[0], num_features)
13         )
14
15         # Cluster assignment layer
16         self.cluster_layer = nn.Parameter(
17             torch.Tensor(num_clusters, hidden_dims[1]))
18
19     def encode(self, x, edge_index):
20         x = F.relu(self.conv1(x, edge_index))
21         x = F.dropout(x, p=0.5, training=self.training)
22         x = self.conv2(x, edge_index)
23         return x
```

With deep clustering training loop, KL divergence loss, etc.

Critical Misunderstanding:

The actual task is about **classical spectral clustering**, not deep graph neural networks. The notebook requires:

1. **Q2: Adjacency Matrix** — Compute weighted adjacency using Gaussian kernel:

$$A_{i,j} = e^{-\gamma \|x_i - x_j\|^2}$$

Reference:

```
1 def get_adjacency_matrix(gamma, X):
2     return np.exp(-gamma * distance.cdist(X, X, metric='sqeuclidean'))
```

2. **Q3: Degree Matrix** — Column sums of adjacency matrix:

```
1 def get_degree_matrix(adjacency_matrix):
2     degree_matrix = np.sum(adjacency_matrix, axis=1)
3     return degree_matrix
```

3. **Q4: Normalized Laplacian** — Compute $M = D^{-1/2}AD^{-1/2}$:

```
1 M = np.multiply(np.sqrt(1/degree_matrix)[np.newaxis, :],
2                 np.multiply(adjacency_matrix,
3                             np.sqrt(1/degree_matrix[:, np.newaxis])))
```

4. Q5: Spectral Clustering — SVD + K-Means on top eigenvectors:

```

1 def cluster_data(M, num_clusters=3):
2     U = svd(M, full_matrices=False, lapack_driver='gesvd')[0]
3     Usub = U[:, :num_clusters]
4     y_pred = KMeans(n_clusters=num_clusters, random_state=seed).fit_predict(
        Usub)
5     return Usub, y_pred

```

5. Q6: Stochastic Matrix — Normalize columns to sum to 1:

```

1 def stochastic_matrix_converter(M):
2     M_stoch = M / np.sum(M, axis=0)[np.newaxis, :]
3     return M_stoch

```

Windsurf's Errors:

- × Uses PyTorch Geometric's GCNConv — not installed or required
- × Implements a deep learning model when simple NumPy/SciPy suffices
- × No adjacency matrix computation from pairwise distances
- × No degree matrix, no normalized Laplacian
- × No SVD-based spectral embedding
- × Completely misses the educational point about why graph representations help clustering

5.1 Graph Clustering Summary

Task	Status	Would Pass Tests?
Adjacency matrix (Q2)	× Not attempted	No
Degree matrix (Q3)	× Not attempted	No
Normalized Laplacian (Q4)	× Not attempted	No
Spectral clustering (Q5)	× Wrong approach	No
Stochastic matrix (Q6)	× Not attempted	No
Normalized K-Means (Q7)	× Not attempted	No

6 Overall Assessment

6.1 Quantitative Summary

Notebook	Correct	Partial	Incorrect
Autoencoders	5	3	0
RNN & Gradients	3	1	1 (+1 missing)
RNN Last Name	0	0	5
Graph Clustering	0	0	6
Total	8	4	12

6.2 Key Observations

1. **Autoencoders (Best Performance):** Windsurf demonstrated strong understanding of autoencoder architectures, correctly implementing the core components. Minor issues with loss function semantics and visualization formatting.
2. **RNN Fundamentals (Mixed):** Core RNN logic was correct, but implementation details (parameter structure) would cause test failures. The gradient visualization task showed partial understanding.
3. **Task Misinterpretation (Critical Failures):**
 - For the name classification task, Windsurf implemented a *generative* model instead of a *discriminative* classifier.
 - For graph clustering, Windsurf proposed a deep learning solution when the task explicitly required classical spectral clustering algorithms.
4. **Context Sensitivity:** Windsurf performed better on tasks with clear, self-contained descriptions (autoencoder methods) but struggled when the task required understanding the broader notebook context or reading detailed specifications.